

numpy.ufunc.reduceat

method

`ufunc.reduceat(array, indices, axis=0, dtype=None, out=None)`

Performs a (local) reduce with specified slices over a single axis.

For `i` in `range(len(indices))`, `reduceat` computes `ufunc.reduce(array[indices[i]:indices[i+1]])`, which becomes the `i`-th generalized “row” parallel to `axis` in the final result (i.e., in a 2-D array, for example, if `axis = 0`, it becomes the `i`-th row, but if `axis = 1`, it becomes the `i`-th column). There are three exceptions to this:

- when `i = len(indices) - 1` (so for the last index), `indices[i+1] = array.shape[axis]`.
- if `indices[i] >= indices[i + 1]`, the `i`-th generalized “row” is simply `array[indices[i]]`.
- if `indices[i] >= len(array)` or `indices[i] < 0`, an error is raised.

The shape of the output depends on the size of `indices`, and may be larger than `array` (this happens if `len(indices) > array.shape[axis]`).

Parameters: `array` : *array_like*

The array to act on.

`indices` : *array_like*

Paired indices, comma separated (not colon), specifying slices to reduce.

`axis` : *int, optional*

The axis along which to apply the reduceat.

`dtype` : *data-type code, optional*

The type used to represent the intermediate results. Defaults to the data type of the output array if this is provided, or the data type of the input array if no output array is provided.

`out` : *ndarray, None, or tuple of ndarray and None, optional*

A location into which the result is stored. If not provided or `None`, a freshly-allocated array is returned. For consistency with `ufunc.__call__`, if given as a keyword, this may be wrapped in a 1-element tuple.

Changed in version 1.13.0: Tuples are allowed for keyword argument.

Returns: `r` : *ndarray*

The reduced values. If `out` was supplied, `r` is a reference to `out`.

Notes

A descriptive example:

If `array` is 1-D, the function `ufunc.accumulate(array)` is the same as `ufunc.reduceat(array, indices)[::2]` where `indices` is `range(len(array) - 1)` with a zero placed in every other element: `indices = zeros(2 * len(array) - 1, indices[1::2] = range(1, len(array)))`.

Don’t be fooled by this attribute’s name: `reduceat(array)` is not necessarily smaller than `array`.

Examples

To take the running sum of four successive values:

```
>>> np.add.reduceat(np.arange(8), [0,4, 1,5, 2,6, 3,7])[::2]
array([ 6, 10, 14, 18])
```

Search the docs ...

[Array objects](#)

[Array API Standard Compatibility](#)

[Constants](#)

[Universal functions \(ufunc\)](#)

[numpy.ufunc](#)

[numpy.ufunc.nin](#)

[numpy.ufunc.nout](#)

[numpy.ufunc.nargs](#)

[numpy.ufunc.ntypes](#)

[numpy.ufunc.types](#)

[numpy.ufunc.identity](#)

[numpy.ufunc.signature](#)

[numpy.ufunc.reduce](#)

[numpy.ufunc.accumulate](#)

[numpy.ufunc.reduceat](#)

[numpy.linalg.linalg](#)

[numpy.ufunc.outer](#)

[numpy.ufunc.at](#)

[Routines](#)

[Typing\(\[numpy.typing\]\(#\)\)](#)

[Global State](#)

[Packaging\(\[numpy.distutils\]\(#\)\)](#)

[NumPy Distutils - Users Guide](#)

[Status of \[numpy.distutils\]\(#\) and
migration advice](#)

[NumPy C-API](#)

[CPU/SIMD Optimizations](#)

[NumPy and SWIG](#)

A 2-D example:

```
>>> x = np.linspace(0, 15, 16).reshape(4,4)
>>> x
array([[ 0.,  1.,  2.,  3.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.],
       [12., 13., 14., 15.]])
```

```
# reduce such that the result has the following five rows:
# [row1 + row2 + row3]
# [row4]
# [row2]
# [row3]
# [row1 + row2 + row3 + row4]
```

```
>>> np.add.reduceat(x, [0, 3, 1, 2, 0])
array([[12., 15., 18., 21.],
       [12., 13., 14., 15.],
       [ 4.,  5.,  6.,  7.],
       [ 8.,  9., 10., 11.],
       [24., 28., 32., 36.]])
```

```
# reduce such that result has the following two columns:
# [col1 * col2 * col3, col4]
```

```
>>> np.multiply.reduceat(x, [0, 3], 1)
array([[ 0.,  3.],
       [120.,  7.],
       [720., 11.],
       [2184., 15.]])
```

[Previous](#)
[numpy.ufunc.reduce](#)

[Next](#)
[numpy.ufunc.nin](#)